

1. Créez une classe `Forme` avec :
  - une méthode `surface()` renvoyant `o`
  - une méthode `description()` renvoyant `"Forme géométrique"`
2. Créez des classes dérivées :
  - `Rectangle` (attributs : longueur, largeur)
  - `Cercle` (attribut : rayon)

3. Redéfinissez `surface()` pour chaque sous-classe.
4. Testez avec une liste de formes et affichez leur surface et description.

#### **Exercice 4 :**

1. Créez une classe `Personne` (nom, âge, méthode `afficher_infos()`).
2. Créez une classe `Employe` (salaire, poste).
3. Créez une classe `Formateur` (matière\_enseignée).
4. Créez une classe `FormateurPermanent` héritant de `Employe` et `Formateur`.
  - Redéfinissez `afficher_infos()` pour inclure toutes les informations.
5. Testez le comportement et le MRO (method resolution order).

#### **Exercice 5 :**

1. Créez une classe `CollectionFormes` qui contient une liste d'objets `Forme`.
2. Rendez-la **itérable** (en implémentant `__iter__` et `__next__`).
3. Testez le polymorphisme en parcourant la collection et en appelant `surface()` sur chaque forme, sans se soucier de leur type.

#### **Exercice 6 :**

1. Créez une classe `Vecteur2D(x, y)`.
2. Implémentez :
  - `__add__` pour additionner deux vecteurs,
  - `__sub__` pour leur différence,
  - `__eq__` pour tester l'égalité,
  - `__str__` pour un affichage clair.
3. Ajoutez une méthode `norme()` pour calculer la norme du vecteur.
4. Testez vos opérations avec plusieurs instances.

#### **Exercice 7 :**

1. Créez une classe abstraite `Employe` avec :
  - attributs : nom, age
  - méthode abstraite `calculer_salaire()`
2. Créez des sous-classes :
  - `EmployeHoraire` (salaire = taux\_horaire × heures)
  - `EmployeMensuel` (salaire = salaire\_fixe)
3. Créez une liste mixte d'employés et affichez le salaire de chacun.